Parte 4: Laboratorio 4: Programación de Planos de Datos con P4

Validación académica – ARP y control de sesiones TCP en el plano de datos

Convención de colores

A lo largo de este documento se utiliza la siguiente convención cromática para distinguir el tipo de terminal o herramienta de cada comando:

- Verde: comandos ejecutados en el shell Bash de la VM Linux (host de laboratorio).
- Azul: código Python o fragmentos de script.
- Rojo: comandos ejecutados dentro de la CLI de Mininet o en terminales de hosts virtuales.
- Naranja: fragmentos de código P4 del programa compilado.

Introducción

Este documento presenta la validación académica del laboratorio Lab Practice 4: Introduction to P4 Programming del curso Internet del Futuro, centrado en la implementación de funcionalidades de ARP y control de sesiones TCP directamente en el plano de datos mediante P4 sobre el switch de referencia BMv2 y Mininet [7].

El laboratorio se organiza en tres componentes principales: (i) un ejercicio guiado de conmutación L2 que introduce la herramienta y el flujo de trabajo de compilación y despliegue de P4 en BMv2, (ii) una actividad donde el switch P4 actúa como ARP responder en el plano de datos, y (iii) una actividad de

control de sesiones TCP en la que el switch descarta selectivamente los intentos de conexión no autorizados en el propio plano de datos [1]. En conjunto, estos ejercicios permiten que el estudiante pase de la comprensión clásica de ARP/TCP su realización explícita como lógica de parsing, tablas y acciones dentro de un pipeline P4 programable [2].

Configuración del entorno de desarrollo P4 (sin ONOS)

Descripción del entorno

Los ejercicios de este laboratorio utilizan el compilador p4c, el switch de referencia BMv2 (simple_switch) y Mininet como emulador de red. A diferencia de otros entornos que incorporan ONOS o Stratum como controladores, el plano de control se limita a la interfaz simple_switch_CLI, lo que permite enfocarse en el programa P4 y en el comportamiento del plano de datos [7].

Se presentan dos rutas: la Opción A parte de una imagen de máquina virtual ya construida con todas las herramientas preinstaladas (recomendada si no se dispone de entorno previo), y la Opción B describe la instalación manual sobre una VM con Ubuntu 20.04 ya disponible.

Nota sobre versiones del sistema operativo: Ubuntu 20.04 LTS es la única versión compatible garantizada para este laboratorio. Ubuntu 22.04 y versiones superiores presentan incompatibilidades conocidas con las dependencias de BMv2 y con los paquetes del PPA oficial de p4lang, que pueden impedir la compilación del programa P4 o el arranque del switch de referencia.

Requisitos del sistema

- Software de virtualización: VirtualBox, VMware u otro compatible con imágenes .ova
- Para la Opción A: mínimo 20 GB de espacio libre para descargar la imagen preconfigurada
- Para la Opción B: imagen ISO de Ubuntu 20.04 LTS y conexión a Internet

- Mínimo 2 núcleos de CPU (se recomiendan 4)
- Mínimo 4 GB de RAM (se recomiendan 8 GB)

Opción A: imagen preconfigurada de Ubuntu 20.04 con herramientas P4

Andy Fingerhut mantiene imágenes de máquina virtual con Ubuntu 20.04 preconfiguradas con p4c, simple_switch, simple_switch_CLI y Mininet, listas para importar directamente en el software de virtualización. Las imágenes están disponibles en la siguiente página: https://github.com/jafingerhut/p4-guide/blob/master/bin/older-images.md

Pasos de instalación (Opción A)

1. Acceder a la página anterior y descargar el archivo .ova correspondiente a Ubuntu 20.04 con p4c y BMv2.
2. Abrir el software de virtualización e importar la imagen descargada. En VirtualBox: Archivo → Importar servicio virtualizado → seleccionar el archivo .ova.
3. Iniciar la VM. Las credenciales por defecto son usuario p4 y contraseña p4.
4. Abrir una terminal dentro de la VM y verificar que las herramientas están disponibles:

```
p4@p4vm:~$ p4c --version
p4@p4vm:~$ simple_switch --version
p4@p4vm:~$ sudo mn --version
```

Si los tres comandos devuelven sus versiones sin error, el entorno está listo. Continúa al apartado de Preparación y primera prueba con el Ejercicio 1.

Opción B: Instalación manual en Ubuntu 20.04 existente

Esta opción configura el entorno P4 sobre cualquier VM o instalación de Ubuntu 20.04 ya disponible.

Paso 1 – Actualizar el sistema e instalar herramientas base

```
usuario@vm:~$ sudo apt-get update
usuario@vm:~$ sudo apt-get upgrade -y
usuario@vm:~$ sudo apt-get install -y git curl wget
```

Paso 2 – Instalar p4c y BMv2 desde el PPA oficial de p4lang

El compilador p4c (p4lang/p4c) y el switch BMv2 (p4lang/behavioral-model) se distribuyen a través del PPA oficial de p4lang para Ubuntu 20.04 [7]:

```
usuario@vm:~$ sudo add-apt-repository ppa:p4lang/ppa
usuario@vm:~$ sudo apt-get update
usuario@vm:~$ sudo apt-get install -y p4lang-p4c p4lang-bmv2
```

El paquete p4lang-p4c instala el compilador p4c y el backend p4c-bm2-ss para BMv2. El paquete p4lang-bmv2 instala simple_switch y simple_switch_CLI.

Verificar la instalación:

```
usuario@vm:~$ p4c --version
usuario@vm:~$ simple_switch --version
usuario@vm:~$ which simple_switch_CLI
```

Paso 3 – Instalar Mininet

```
usuario@vm:~$ sudo apt-get install -y mininet
usuario@vm:~$ sudo mn --version
```

Para este laboratorio no se necesita ningún controlador OpenFlow (ONOS, POX ni ovs-testcontroller). El switch P4 es controlado directamente por sim-

ple_switch_CLI; Mininet se usa únicamente como emulador de topología de red.

Fundamentación teórica

P4 y planos de datos programables

P4 es un lenguaje de dominio específico diseñado para especificar cómo los dispositivos del plano de datos (conmutadores, routers, NICs programables) procesan paquetes, enfatizando tres propiedades clave: reconfigurabilidad, independencia de protocolo e independencia de target [1]. Un programa P4 describe explícitamente las cabeceras que se reconocen, el parser, las tablas de coincidencia (match-action), las acciones asociadas y el flujo de control del pipeline (controles de ingreso y egreso, deparser), normalmente sobre arquitecturas como v1model o PSA [1].

En este laboratorio, el target software es BMv2 (Behavioral Model v2), el switch de referencia de P4, ejecutado mediante la variante simple_switch integrada con Mininet [7]. BMv2 se controla a través de un archivo JSON generado por el compilador p4c y tablas pobladas en tiempo de ejecución mediante simple_switch_CLI, lo que materializa la separación entre la descripción estática del pipeline y la configuración dinámica de las reglas de reenvío [7]. Conceptualmente, esto ilustra el principio de target independence: el mismo programa P4 puede compilarse a diferentes plataformas (BMv2 software, Tofino ASIC, NetFPGA) simplemente cambiando el compilador backend, sin modificar el código fuente P4 [2].

Protocolo ARP y su realización en P4

El protocolo ARP (Address Resolution Protocol) está especificado en la RFC 826 [3] como el mecanismo estándar para mapear direcciones de protocolo (p. ej. IP) a direcciones de enlace (p. ej. MAC Ethernet) mediante mensajes de solicitud y respuesta que incluyen campos como h_type, p_type, oper, src_mac, src_ip, dst_mac y dst_ip. Operacionalmente, un host que conoce la IP de destino pero no su MAC emite una solicitud ARP en broadcast; el host destino

responde con una trama unicast que permite al origen completar su tabla ARP y transmitir los paquetes IP subsiguientes.

Implementar un ARP responder en P4 exige modelar la cabecera ARP como un tipo de cabecera, extender el parser Ethernet para reconocer el EtherType 0x0806, declarar una tabla arp_exact con clave sobre el campo dst_ip y una acción arp_reply que construya la respuesta intercambiando las direcciones y devuelva el paquete por el mismo puerto de ingreso [1]. Desde el punto de vista conceptual, esto ejemplifica cómo un protocolo bien definido en RFC se traduce a estructuras de cabecera y lógica de control dentro de un pipeline P4, desplazando funcionalidad tradicionalmente residente en los hosts hacia el propio switch de reenvío [2].

Tráfico TCP y control de sesiones en el plano de datos

TCP está especificado en la RFC 793 [4] como un protocolo orientado a conexión que provee un flujo fiable de bytes sobre IP, con una cabecera que incluye puertos de origen y destino, números de secuencia y acuse de recibo, flags de control (SYN, ACK, FIN, RST, etc.), ventana y otros campos auxiliares. El establecimiento de conexión utiliza un proceso de negociación de tres vías (SYN, SYN-ACK, ACK), cuya seguridad puede reforzarse controlando qué flujos tienen permiso para completar el handshake.

En un plano de datos programable, la cuádrupla (IP origen, IP destino, puerto origen, puerto destino) puede usarse como clave de clasificación para implementar tablas de autorización de sesiones. En este laboratorio, el switch P4 mantiene una tabla tcp_sessions indexada por dicha cuádrupla: los paquetes SYN pertenecientes a flujos registrados se reenvían normalmente, mientras que los SYN no registrados son descartados silenciosamente en el plano de datos antes de alcanzar el host destino [6]. Esto permite implementar una forma básica de control de acceso a nivel de transporte directamente en el switch, sin intervención del plano de control para cada paquete [2].

Relación con SDN y separación control/datos

El paradigma SDN formalizado en la RFC 7426 [5] describe una separación clara entre el plano de control, responsable de decidir las políticas y rutas, y el plano de datos, encargado de ejecutar dichas decisiones mediante operaciones de filtrado, clasificación y reenvío de paquetes. Mientras que OpenFlow fija un conjunto predefinido de campos y tablas, P4 extiende este modelo permitiendo que la propia semántica del pipeline sea programable, integrando descriptores de cabecera y tablas arbitrarias que luego son gestionadas por el plano de control a través de interfaces como P4Runtime [1].

En este laboratorio, las reglas se instalan mediante simple_switch_CLI, lo que corresponde a una interfaz de control simplificada que actúa sobre el datapath descrito por el programa P4 compilado a JSON. El estudiante observa cómo el plano de control (scripts o CLI) rellena tablas que el plano de datos aplica a cada paquete, reforzando la comprensión de la separación entre la descripción del comportamiento (código P4) y la configuración de política (entradas de tabla en tiempo de ejecución) [5].

Descripción general del laboratorio

La guía original Lab Practice 4: Introduction to P4 Programming se ejecuta sobre una topología sencilla en Mininet con un único switch P4 (s1) y dos hosts conectados (h1 y h2), utilizando BMv2 como switch software y el compilador p4c-bm2-ss como backend para generar el archivo bmv2.json y el descriptor p4info.txt [7]. El flujo básico consiste en compilar el programa main.p4, iniciar la topología a través de un script de Mininet y, finalmente, cargar las tablas de reenvío con simple_switch_CLI usando el archivo de comandos s1-commands.txt.

El laboratorio está estructurado en tres bloques: (1) un ejercicio guiado de conmutación L2 que utiliza una tabla l2_exact_table para mapear direcciones MAC destino a puertos de salida, (2) una actividad ARP donde se elimina la población estática de tablas ARP en los hosts y el switch P4 responde direc-

tamente a las solicitudes ARP en el plano de datos, y (3) una actividad de control de sesiones TCP donde el switch sólo permite conexiones cuyo flujo ha sido previamente registrado en una tabla tcp_sessions. En lo que sigue, se presenta la validación pedagógica y técnica paso a paso de cada componente, con especial énfasis en las funciones de ARP y TCP solicitadas en el plano de datos programable [1].

La topología utilizada en los tres ejercicios es la siguiente:

- **h1**: dirección IP 10.0.0.1, dirección MAC 00:00:00:00:00:01, conectado al puerto 1 del switch.
- **h2**: dirección IP 10.0.0.2, dirección MAC 00:00:00:00:00:02, conectado al puerto 2 del switch.
- **s1**: switch BMv2 escuchando en el puerto Thrift 9090, cargado con el pipeline P4 compilado.

Los enlaces están configurados con parámetros de QoS: el enlace h1–s1 tiene 2 Mbit/s de ancho de banda, 10 ms de retardo y 5 % de pérdida; el enlace h2–s1 tiene 5 Mbit/s de ancho de banda, 1 ms de retardo y 2 % de pérdida.

Validación del Ejercicio 1 – Conmutador L2 sobre P4

El ejercicio 1 es fundamental porque introduce el ciclo de desarrollo P4–BMv2–Mininet y verifica el correcto funcionamiento del pipeline base de reenvío L2, sobre el cual se construyen las actividades posteriores [8]. El programa main.p4 para este ejercicio declara una cabecera Ethernet (ethernet_t), un parser que extrae dicha cabecera, un control de ingreso que aplica una tabla l2_exact_table con coincidencia exacta sobre hdr.ethernet.dst_addr y una acción set_egress_port que fija el puerto de salida en función de la MAC destino. En egreso, se añade una cabecera de metadatos opcional (cpu_in) para marcar paquetes dirigidos al controlador, y el deparser emite únicamente las cabeceras válidas, reensamblando el paquete antes de su transmisión [7].

Preparación y primera prueba con el Ejercicio 1

Una vez instalado el entorno por cualquiera de las opciones anteriores, la estructura de archivos requerida para el Ejercicio 1 es la siguiente:

```
P4_Labs_Docs/
 1/
     p4src/
        main.p4
     mininet/
        topo.py
        p4_mininet.py
     s1-commands.txt
```

A continuación se presentan los fragmentos clave del programa P4 del ejercicio 1:

```
header ethernet_t {
  macAddr_t  dst_addr;
  macAddr_t  src_addr;
  bit<16>  ether_type;
}
```

```
state parse_ethernet {
  packet.extract(hdr.ethernet);
  transition parse_ipv4;
}
```

```
table l2_exact_table {
  key = { hdr.ethernet.dst_addr: exact; }
  actions = { set_egress_port; @defaultonly drop; }
}
```

Pasos de validación del ejercicio 1

1. **Compilación del programa P4**

Objetivo: Generar los artefactos bmv2.json y p4info.txt que describen el pipeline P4 para el switch BMv2.

Concepto de red implicado: Independencia de target en P4 y separación entre especificación de alto nivel y representación compilada específica para BMv2. El compilador p4c-bm2-ss toma el programa main.p4, que importa la arquitectura v1model, y lo traduce a una representación intermedia en JSON que BMv2 interpreta en tiempo de ejecución [7].

Comandos ejecutados:

```
P4_Labs_Docs/1# mkdir -p p4src/build
```

```
P4_Labs_Docs/1# p4c-bm2-ss –p4v 16 -o p4src/build/bmv2.json \
  –p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

Salida obtenida:

```
[--Wwarn=deprecated] warning: .txt format is being
deprecated; use .txtpb instead
```

Análisis de la salida: El compilador finaliza sin errores. La advertencia sobre el formato .txt es esperada para esta versión del compilador y puede ignorarse; el archivo bmv2.json se genera correctamente en p4src/build/. Este aviso indica que el formato de descripción de P4Runtime está siendo migrado de texto plano a protobuf text format (.txtpb), pero no afecta al funcionamiento del switch BMv2 en el contexto de este laboratorio.

Explicación pedagógica: Este paso permite al estudiante interiorizar que P4 no se ejecuta directamente, sino que es compilado a una descripción específica para un dispositivo objetivo (en este caso BMv2), reforzando la idea de separación entre el lenguaje y la plataforma de ejecución, análogamente a como un programa en C se compila a código máquina para una arquitectura concreta [2].

2. **Inicio de la topología en Mininet**

Objetivo: Instanciar la topología con dos hosts y el switch P4 (s1) y arrancar BMv2 con el archivo JSON generado.

Concepto de red implicado: Emulación de redes mediante Mininet y acoplamiento entre topologías virtuales y switches programables basados en software. El script topo.py emplea la clase P4Switch (definida en p4_mininet.py) que lanza el proceso simple_switch apuntando al archivo bmv2.json y configura el puerto Thrift para la instalación posterior de reglas [7].

Comando ejecutado:

P4_Labs_Docs/1# python3 mininet/topo.py

Salida obtenida:

```
*** Creating network
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(2.00Mbit 10ms delay 5.00000% loss) (h1, s1)
(5.00Mbit 1ms delay 2.00000% loss) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 Starting s1 @ thrift-port 9090

*** Starting CLI:
mininet>
```

Análisis de la salida: Mininet crea exitosamente la red virtual con los dos hosts y el switch P4. Se muestran los parámetros de QoS configurados en los enlaces: el enlace h1–s1 opera a 2 Mbit/s con 10 ms de retardo y 5 % de pérdida, mientras que el enlace h2–s1 opera a 5 Mbit/s con 1 ms de retardo y 2 % de pérdida. Estas restricciones permiten observar el impacto

de las condiciones del enlace en las pruebas de conectividad. La CLI de Mininet queda disponible en el terminal actual; se mantiene activa para el resto del ejercicio [7].

Explicación pedagógica: Este paso vincula el modelo lógico del pipeline P4 con un entorno de pruebas realista donde se pueden generar y observar tramas. El hecho de que el switch no aplique ningún comportamiento hasta que se instalen las reglas refuerza la idea de separación entre la descripción del pipeline (el programa P4) y la política de reenvío (las entradas de tabla instaladas en tiempo de ejecución) [5].

3. **Población de la tabla L2 mediante simple_switch_CLI**

   Objetivo: Instalar las entradas de reenvío L2 que mapean las direcciones MAC de los hosts a los puertos correspondientes del switch.

   Concepto de red implicado: Tablas match-action en el plano de datos y separación entre pipeline programado y política de reenvío definida en tiempo de ejecución. Cada línea del archivo s1-commands.txt sigue la sintaxis table_add ¡tabla¿¡acción¿¡clave¿=¿¡parámetros¿, añadiendo entradas a l2_exact_table cuya clave es la dirección MAC destino y cuyo parámetro es el número de puerto de salida [1].

   Contenido de s1-commands.txt para el ejercicio 1:

```
table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:01 => 1

table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:02 => 2
```

   Comando ejecutado (en segundo terminal):

```
P4_Labs_Docs/1# simple_switch_CLI –thrift-port 9090 < s1-commands.txt
```

   Salida obtenida:

```
Obtaining JSON from switch...
Done
```

```
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:01
action:              IngressPipeImpl.set_egress_port
runtime data:        00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:02
action:              IngressPipeImpl.set_egress_port
runtime data:        00:02
Entry has been added with handle 1
RuntimeCmd:
```

Análisis de la salida: La CLI se conecta al switch mediante el protocolo Thrift en el puerto 9090 y añade dos entradas a l2_exact_table: la MAC 00:00:00:00:00:01 (h1) se mapea al puerto 1 (dato de tiempo de ejecución 00:01) y la MAC 00:00:00:00:00:02 (h2) se mapea al puerto 2. Los manejadores (handles) 0 y 1 son identificadores internos de BMv2 asignados a cada entrada en el orden de inserción.

Explicación pedagógica: Este paso evidencia el rol del plano de control (aquí representado por la CLI) para poblar las tablas definidas en P4. Es importante notar que, a diferencia de un switch con aprendizaje dinámico de MACs, en este pipeline P4 el reenvío L2 es completamente estático y determinista: no hay ningún algoritmo automático de aprendizaje, toda la lógica está explícitamente definida en el programa y en las entradas instaladas [5].

4. **Verificación de conectividad mediante pingall**

   Objetivo: Confirmar que el switch P4 reenvía correctamente tramas entre h1 y h2 usando la tabla L2 poblada.

   Concepto de red implicado: Pruebas de conectividad mediante ICMP y verificación de reenvío L2 determinista en un pipeline P4. En la confi-

guración de este ejercicio, topo.py llama a net.staticArp() para pre-poblar las tablas ARP de los hosts, eliminando la necesidad de resolución ARP y permitiendo que las pruebas de conectividad validen exclusivamente el reenvío L2.

Comando ejecutado (en la CLI de Mininet):

mininet> pingall

Salida obtenida:

```
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

Análisis de la salida: Ambas pruebas son exitosas (0 % de pérdida en el nivel de accesibilidad). Dado que los parámetros de QoS incluyen pérdidas del 5 % y 2 % en los respectivos enlaces, es normal que algunos paquetes ICMP individuales se pierdan en sucesivos ping -c N, pero el pingall verifica únicamente accesibilidad. El resultado demuestra que las entradas de l2_exact_table son correctas y que el pipeline P4 reenvía tramas Ethernet en función de la dirección MAC destino según lo especificado.

Explicación pedagógica: Al comparar este comportamiento con el de un switch OpenFlow clásico con controlador reactivo, el estudiante aprecia que en P4 el comportamiento L2 es resultado directo de un programa y de la política instalada previamente; no existe ningún mecanismo de aprendizaje implícito en el hardware. Toda la lógica es observable e inspeccionable en el código fuente P4, lo que facilita la verificación formal y el razonamiento sobre el comportamiento del switch [8].

Actividad 1 – Ejercicio 2: Respuesta ARP en el switch P4

En el ejercicio 2 se elimina la llamada net.staticArp() del script de Mininet para evitar que los hosts inicien con tablas ARP pre-pobladas, de modo que cualquier comunicación IP entre h1 y h2 requiera el intercambio estándar de solicitudes y respuestas ARP. El objetivo es que el switch P4 implemente un in-switch ARP responder: al recibir una solicitud ARP, el switch genera la respuesta directamente en el plano de datos y la devuelve por el mismo puerto de ingreso, sin reenviar ningún tráfico ARP al host destino [3].

Cambios estructurales en el programa P4

Los TODO del ejercicio 2 abarcan los siguientes componentes del programa main.p4:

- Definición de constantes: EtherType ARP (0x0806), campos del protocolo ARP conforme a RFC 826 (ARP_HTYPE, ARP_PTYPE, ARP_HLEN, ARP_PLEN, ARP_REQ, ARP_REPLY).
- Cabecera arp_t con los campos de la PDU ARP e inclusión en la estructura parsed_headers_t.
- Extensión del parser: el estado parse_ethernet ramifica según el EtherType hacia parse_arp o parse_ipv4.
- Acción arp_reply en el control de ingreso y tabla arp_exact indexada por hdr.arp.dst_ip.
- Bloque apply: lógica condicional para enrutar tráfico ARP a arp_exact y tráfico IPv4 a l2_exact_table.
- Deparser: emisión de la cabecera ARP cuando es válida.

Pasos de validación de la actividad ARP

1. **Paso 1 – Definición de la cabecera ARP y constantes**

Objetivo: Modelar la PDU ARP como tipo de cabecera P4 alineado con la especificación de la RFC 826 y definir las constantes necesarias para el procesamiento en el pipeline.

Concepto de red implicado: Correspondencia exacta entre los campos de la cabecera ARP en el wire-format de la RFC 826 y su representación como campos P4 con anchuras específicas en bits. La PDU ARP sobre Ethernet/IPv4 tiene siempre la misma estructura fija: 28 bytes con campos de tipo hardware (16 bits), tipo protocolo (16 bits), longitud de dirección HW (8 bits), longitud de dirección protocolo (8 bits), operación (16 bits), y los cuatro campos de dirección (MAC 48 bits + IP 32 bits, repetidos para origen y destino) [3].

Fragmento P4 implementado:

```
const bit<16>ETHERTYPE_ARP = 0x0806;
const bit<16>ARP_HTYPE = 0x0001;
const bit<16>ARP_PTYPE = 0x0800;
const bit<8>ARP_HLEN = 6;
const bit<8>ARP_PLEN = 4;
const bit<16>ARP_REQ = 1;
const bit<16>ARP_REPLY = 2;

header arp_t {
  bit<16>h_type;
  bit<16>p_type;
  bit<8>h_len;
  bit<8>p_len;
  bit<16>oper;
  mac_addr_t src_mac;
  ipv4_addr_t src_ip;
  mac_addr_t dst_mac;
  ipv4_addr_t dst_ip;
}
```

Comportamiento esperado: El compilador acepta la definición de cabecera y las constantes. Las instancias de hdr.arp quedan disponibles para

el parser y el control de ingreso cuando el campo ether_type de la trama Ethernet sea 0x0806.

Explicación pedagógica: Este paso ilustra el principio fundamental de P4: todo protocolo procesado en el pipeline debe ser modelado explícitamente como cabecera. La correspondencia campo a campo con la RFC refuerza en el estudiante la habilidad de leer especificaciones de protocolo y traducirlas a tipos de datos en un lenguaje de dominio específico [2].

2. **Paso 2 – Extensión del parser: estados parse_ethernet y parse_arp**

   Objetivo: Hacer que el parser distinga entre tráfico IPv4 y ARP basándose en el campo ether_type, extrayendo la cabecera ARP cuando sea necesario.

   Concepto de red implicado: Parsing jerárquico de cabeceras y ramificación del pipeline en función de tipos de protocolo en la capa de enlace. El parser de P4 es esencialmente un autómata de estados finitos: cada estado extrae una cabecera y decide el siguiente estado basándose en el valor de uno o más campos [1].

   Fragmento P4 implementado:

```
state parse_ethernet {
  packet.extract(hdr.ethernet);
  transition select(hdr.ethernet.ether_type) {
    ETHERTYPE_IPV4: parse_ipv4;
    ETHERTYPE_ARP: parse_arp;
    default: accept;
  }
}

state parse_arp {
  packet.extract(hdr.arp);
  transition accept;
}
```

   Comportamiento esperado: Cuando llega una trama con EtherType 0x0806, el parser marca hdr.arp como válida y deja disponibles todos sus campos

para las tablas de ingreso. Las tramas IPv4 siguen el camino original hacia parse_ipv4 sin verse afectadas.

Explicación pedagógica: Este paso ayuda al estudiante a ver el parser P4 como la instancia concreta de un autómata programable que decide qué cabeceras serán visibles para el resto del pipeline, en contraste con los pipelines fijos de switches tradicionales donde el conjunto de cabeceras reconocidas está determinado por el fabricante [2].

3. **Paso 3 – Implementación de la acción arp_reply**

   Objetivo: Programar una acción de ingreso que construya una respuesta ARP intercambiando las direcciones origen/destino y actualizando el campo de operación a REPLY, devolviendo el paquete modificado por el mismo puerto de entrada.

   Concepto de red implicado: Semántica de la respuesta ARP según RFC 826: la respuesta debe contener como dirección de protocolo destino la IP origen de la solicitud y como dirección de hardware destino la MAC origen de la solicitud, con la MAC del nodo que responde como dirección de hardware origen [3].

   Fragmento P4 implementado:

```
action arp_reply(mac_addr_t request_mac) {
  hdr.arp.oper = ARP_REPLY;
  hdr.arp.dst_mac = hdr.arp.src_mac;
  hdr.arp.src_mac = request_mac;
  // Intercambio de IPs usando campo temporal
  local_metadata.arp_tmp_ip = hdr.arp.src_ip;
  hdr.arp.src_ip = hdr.arp.dst_ip;
  hdr.arp.dst_ip = local_metadata.arp_tmp_ip;
  hdr.ethernet.dst_addr = hdr.ethernet.src_addr;
  hdr.ethernet.src_addr = request_mac;
  standard_metadata.egress_spec =
    standard_metadata.ingress_port;
}
```

Análisis del fragmento: La acción realiza 8 asignaciones secuenciales. El intercambio de las direcciones IP debe usar un campo temporal (arp_tmp_ip) porque en P4 las asignaciones son secuenciales y, de lo contrario, la primera asignación sobreescribiría el valor original de src_ip antes de que pudiera usarse en la segunda. Finalmente, egress_spec se fija al valor de ingress_port para devolver la respuesta por el mismo enlace por el que llegó la solicitud.

Explicación pedagógica: El estudiante observa que es posible generar paquetes de respuesta directamente en el switch mediante acciones P4, internalizando que la lógica del plano de datos puede ir más allá del simple reenvío transparente de tramas, actuando como un agente activo de procesamiento y generación de tráfico de control [6].

4. **Paso 4 – Tabla arp_exact y lógica en el bloque apply**

   Objetivo: Declarar la tabla arp_exact indexada por la IP objetivo de la solicitud y añadir la ramificación condicional en el bloque apply para separar el tratamiento de ARP e IPv4.

   Concepto de red implicado: Uso de tablas match-action para implementar un ARP proxy controlado por política, donde la MAC de respuesta es provista por el plano de control al instalar las entradas de tabla [5].

   Fragmento P4 implementado:

```
table arp_exact {
  key = { hdr.arp.dst_ip: exact; }
  actions = { arp_reply; drop; }
  default_action = drop();
}

apply {
  if (hdr.arp.isValid()) {
    arp_exact.apply();
  } else {
    l2_exact_table.apply();
    acl_table.apply();
```

```
    }
}
```

Contenido de s1-commands.txt para el ejercicio 2:

```
table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:01 => 1

table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:02 => 2

table_add IngressPipeImpl.arp_exact
IngressPipeImpl.arp_reply 10.0.0.1 => 00:00:00:00:00:01

table_add IngressPipeImpl.arp_exact
IngressPipeImpl.arp_reply 10.0.0.2 => 00:00:00:00:00:02
```

Análisis del contenido: Las dos primeras entradas replican el reenvío L2 del ejercicio 1. Las dos nuevas entradas de arp_exact mapean la IP destino de cada solicitud ARP a la MAC correspondiente del host: cuando el switch recibe who-has 10.0.0.1, consulta la tabla con clave 10.0.0.1 y obtiene la MAC 00:00:00:00:00:01 como parámetro de la acción arp_reply.

Explicación pedagógica: Este paso vincula el concepto de tablas P4 con el patrón de diseño de ARP proxy, mostrando cómo la política de resolución de direcciones puede modificarse dinámicamente insertando, actualizando o eliminando entradas de tabla sin recompilar el programa P4 [5].

5. **Paso 5 – Compilación, inicio de topología e instalación de reglas**

   Objetivo: Repetir el flujo de trabajo de compilación y despliegue aplicando el programa P4 extendido con ARP.

   Comandos ejecutados:

```
P4_Labs_Docs/2# mkdir -p p4src/build
P4_Labs_Docs/2# p4c-bm2-ss –p4v 16 -o p4src/build/bmv2.json \
    –p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

P4_Labs_Docs/2# simple_switch_CLI –thrift-port 9090 < s1-commands.txt

Salida obtenida (compilación e instalación de reglas):

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:01
action:              IngressPipeImpl.set_egress_port
runtime data:        00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:02
action:              IngressPipeImpl.set_egress_port
runtime data:        00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.arp_exact
match key:           EXACT-0a:00:00:01
action:              IngressPipeImpl.arp_reply
runtime data:        00:00:00:00:00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.arp_exact
match key:           EXACT-0a:00:00:02
action:              IngressPipeImpl.arp_reply
runtime data:        00:00:00:00:00:02
Entry has been added with handle 1
RuntimeCmd:
```

Análisis de la salida: El compilador acepta el programa extendido con ARP sin errores adicionales. La CLI instala las cuatro entradas de tabla en

el orden del archivo: primero las dos entradas de l2_exact_table (handles 0 y 1), luego las dos de arp_exact (handles 0 y 1 en su propia tabla). Las claves de arp_exact se muestran en representación hexadecimal: 0a:00:00:01 corresponde a la IP 10.0.0.1 y 0a:00:00:02 a 10.0.0.2.

Explicación pedagógica: La instalación exitosa de las entradas confirma que el programa P4 define correctamente las tablas y que el plano de control puede poblarlas en tiempo de ejecución sin necesidad de recompilar el pipeline [5].

6. **Paso 6 – Verificación experimental: tcpdump y ping**

   Objetivo: Comprobar que el switch responde a las solicitudes ARP de h1 sin reenviar ningún paquete ARP a h2, y que la conectividad IP entre h1 y h2 funciona una vez resuelta la dirección MAC.

   Comandos ejecutados:

   Desde la CLI de Mininet se abren dos terminales de host:

   mininet> xterm h1 h2

   En la terminal de h2 se captura tráfico ARP en la interfaz:

   (h2)# tcpdump -i h2-eth0 arp -v

   En la terminal de h1 se vacía la tabla ARP y se lanza un ping:

   (h1)# ip neigh flush all
   (h1)# ping -c 1 10.0.0.2

   Salida obtenida en h1 (ping):

```
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=60.0 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 59.968/59.968/59.968/0.000 ms
```

   Salida obtenida en h2 (tcpdump):

```
tcpdump: listening on h2-eth0, link-type EN10MB (Ethernet),
capture size 262144 bytes
02:48:02.208126 ARP, Ethernet (len 6), IPv4 (len 4),
Request who-has 10.0.0.1 tell 10.0.0.2, length 28

02:48:02.210809 ARP, Ethernet (len 6), IPv4 (len 4),
Reply 10.0.0.1 is-at 00:00:00:00:00:01
```

Análisis de la salida: El ping desde h1 hacia h2 tiene éxito con un RTT de 60 ms, coherente con el retardo de 10 ms del enlace h1–s1 más el retardo de 1 ms del enlace s1–h2 (más tiempos de procesamiento de BMv2).

La salida de tcpdump en h2 merece especial atención: las tramas que h2 ve son una solicitud ARP who-has 10.0.0.1 tell 10.0.0.2 y su respución 10.0.0.1 is-at 00:00:00:00:00:01. Esto revela que h2 también necesita resolver la MAC de h1 para responder a los paquetes ICMP que le llegan. La solicitud ARP de h2 tampoco llega a h1: el switch la intercepta e inyecta la respuesta directamente, de modo que h1 nunca ve esta solicitud. El resultado clave es que el tcpdump de h2 no muestra las solicitudes ARP originadas por h1 hacia h2: esas tramas son interceptadas y respondidas directamente por el switch sin pasar por el host destino.

Explicación pedagógica: Este resultado hace evidente que funciones tradicionalmente implementadas en los hosts pueden migrarse al plano de datos, reduciendo el tráfico broadcast y la carga del plano de control [3]. La bidireccionalidad de la resolución ARP muestra que ambos hosts resuelven la MAC del otro antes del intercambio IP, y que el switch intercepta ambas solicitudes, respondiendo a cada una desde la tabla arp_exact sin que ningún tráfico ARP atraviese el switch de extremo a extremo [3].

Actividad 2 – Ejercicio 2.2: Control de sesiones TCP en el plano de datos

En el ejercicio 2.2 se extiende el pipeline de ARP e IPv4 para añadir control de sesiones TCP: sólo los flujos cuya cuádrupla (IP origen, IP destino, puerto ori-

gen, puerto destino) ha sido registrada explícitamente en la tabla tcp_sessions son reenviados al host destino. Los paquetes SYN que no corresponden a ninguna sesión registrada son descartados silenciosamente en el plano de datos antes de alcanzar el host destino [6]. Este diseño implementa una forma básica de control de acceso a nivel de transporte sin intervención del plano de control en cada paquete, y sin generar ninguna respuesta TCP hacia el host origen [4].
Cabe resaltar la diferencia con un enfoque de RST activo: el lab original específica un descarte silencioso (silent drop), no la generación de segmentos RST. El script topo.py de este ejercicio añade una regla iptables en cada host para suprimir los RST generados por el kernel de Linux, evitando que interfieran con el filtrado en el plano de datos.

Cambios estructurales en el programa P4

Los TODO del ejercicio 2.2 abarcan:

- Constante IP_PROTO_TCP = 6 y definición de la cabecera tcp_t con todos los campos de la cabecera TCP.
- Extensión del parser IPv4: nuevo estado parse_tcp activado cuando hdr.ipv4.protocol == IP_PROTO_TCP.
- Tabla tcp_sessions indexada por la cuádrupla IP/puerto, con drop como acción por defecto.
- Bloque apply: para tráfico TCP válido, se aplica tcp_sessions; un hit pasa el paquete al reenvío L2, un miss descarta el paquete.
- Deparser: emisión de la cabecera TCP cuando es válida.

Pasos de validación de la actividad TCP

1. **Paso 1 – Definición de la cabecera TCP y extensión del parser**

   Objetivo: Representar la cabecera TCP en P4 y hacer que el parser la extraiga cuando el campo protocol de IPv4 sea 6.

Concepto de red implicado: Estructura de la cabecera TCP según RFC 793 [4]: puertos de origen y destino (16 bits cada uno), número de secuencia (32 bits), número de acuse de recibo (32 bits), data offset (4 bits), reservado (3 bits), flags individuales (9 bits: NS, CWR, ECE, URG, ACK, PSH, RST, SYN, FIN), ventana (16 bits), checksum (16 bits) y puntero urgente (16 bits). La suma de estas anchuras es 160 bits = 20 bytes, que corresponde al encabezado TCP mínimo sin opciones.

Fragmento P4 implementado:

```
const bit<8>IP_PROTO_TCP = 6;

header tcp_t {
  l4_port_t srcPort;
  l4_port_t dstPort;
  bit<32>seqNo;
  bit<32>ackNo;
  bit<4>dataOffset;
  bit<3>res;
  bit<1>ns;
  bit<1>cwr;
  bit<1>ece;
  bit<1>urg;
  bit<1>ack;
  bit<1>psh;
  bit<1>rst;
  bit<1>syn;
  bit<1>fin;
  bit<16>window;
  bit<16>checksum;
  bit<16>urgentPtr;
}

state parse_ipv4 {
  packet.extract(hdr.ipv4);
  transition select(hdr.ipv4.protocol) {
```

```
    IP_PROTO_TCP: parse_tcp;
    default: accept;
  }
}

state parse_tcp {
  packet.extract(hdr.tcp);
  transition accept;
}
```

Comportamiento esperado: Los paquetes TCP llegan al control de ingreso con hdr.tcp válida. Los flags individuales (SYN, ACK, RST, etc.) son accesibles como campos bit a bit, lo que permite implementar lógica de clasificación granular en acciones y tablas.

Explicación pedagógica: El estudiante traduce nuevamente una especificación de RFC a tipos P4, reforzando su habilidad para mapear estructuras de protocolo a representaciones concretas en el pipeline programable. La declaración de flags individuales como campos de 1 bit permite reutilizarlos directamente como claves de tabla o condiciones en el bloque apply [1].

2. **Paso 2 – Tabla tcp_sessions y lógica apply**

   Objetivo: Declarar la tabla de sesiones TCP indexada por la cuádrupla de flujo, con descarte como acción por defecto, e integrar la ramificación correspondiente en el bloque apply.

   Concepto de red implicado: Control de sesiones y filtrado basado en flujos, función típica de firewalls y middleboxes de seguridad, ahora expresada directamente en el plano de datos programable sin intervención del plano de control para cada paquete [6].

   Fragmento P4 implementado:

```
table tcp_sessions {
  key = {
    hdr.ipv4.src_addr: exact;
```

```
    hdr.tcp.srcPort: exact;
    hdr.ipv4.dst_addr: exact;
    hdr.tcp.dstPort: exact;
  }
  actions = { NoAction; drop; }
  default_action = drop();
}

apply {
  if (hdr.arp.isValid()) {
    arp_exact.apply();
  } else if (hdr.tcp.isValid()) {
    if (tcp_sessions.apply().hit) {
      l2_exact_table.apply();
    }
  } else {
    l2_exact_table.apply();
    acl_table.apply();
  }
}
```

Contenido de s1-commands.txt para el ejercicio 2.2:

```
table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:01 => 1

table_add IngressPipeImpl.l2_exact_table
IngressPipeImpl.set_egress_port 00:00:00:00:00:02 => 2

table_add IngressPipeImpl.arp_exact
IngressPipeImpl.arp_reply 10.0.0.1 => 00:00:00:00:00:01

table_add IngressPipeImpl.arp_exact
IngressPipeImpl.arp_reply 10.0.0.2 => 00:00:00:00:00:02
```

```
table_add IngressPipeImpl.tcp_sessions
NoAction 10.0.0.1 5000 10.0.0.2 9999 =>

table_add IngressPipeImpl.tcp_sessions
NoAction 10.0.0.2 9999 10.0.0.1 5000 =>
```

Análisis del contenido: Las cuatro primeras entradas replican el comportamiento de los ejercicios anteriores. Las dos entradas de tcp_sessions registran la sesión bidireccional 10.0.0.1:5000 ↔ 10.0.0.2:9999: una entrada para los paquetes de h1 hacia h2 (origen 10.0.0.1:5000, destino 10.0.0.2:9999) y otra para los paquetes de h2 hacia h1 (origen 10.0.0.2:9999, destino 10.0.0.1:5000). La necesidad de registrar ambas direcciones refleja que la tabla de coincidencia no es bidireccional por defecto [4].

Explicación pedagógica: El estudiante relaciona el concepto abstracto de "sesión TCP" con una implementación concreta basada en tablas P4. La necesidad de dos entradas por sesión (una en cada dirección) evidencia que, a diferencia de los firewalls stateful que crean automáticamente el estado de retorno, este mecanismo P4 es stateless y require entradas explícitas para cada dirección del flujo [6].

3. **Paso 3 – Compilación, inicio de topología e instalación de reglas**

   Objetivo: Compilar el programa P4 extendido con TCP, iniciar la topología y poblar todas las tablas.

   Comandos ejecutados:

```
P4_Labs_Docs/2.2# mkdir -p p4src/build
P4_Labs_Docs/2.2# p4c-bm2-ss –p4v 16 -o p4src/build/bmv2.json \
    –p4runtime-files p4src/build/p4info.txt p4src/main.p4
P4_Labs_Docs/2.2# simple_switch_CLI –thrift-port 9090 ¡s1-commands.txt
```

   Salida obtenida:

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
```

```
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:01
action:              IngressPipeImpl.set_egress_port
runtime data:        00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.l2_exact_table
match key:           EXACT-00:00:00:00:00:02
action:              IngressPipeImpl.set_egress_port
runtime data:        00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.arp_exact
match key:           EXACT-0a:00:00:01
action:              IngressPipeImpl.arp_reply
runtime data:        00:00:00:00:00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.arp_exact
match key:           EXACT-0a:00:00:02
action:              IngressPipeImpl.arp_reply
runtime data:        00:00:00:00:00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.tcp_sessions
match key:           EXACT-0a:00:00:01  EXACT-13:88
EXACT-0a:00:00:02  EXACT-27:0f
action:              NoAction
runtime data:
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table
IngressPipeImpl.tcp_sessions
match key:           EXACT-0a:00:00:02  EXACT-27:0f
```

```
EXACT-0a:00:00:01  EXACT-13:88
action:                NoAction
runtime data:
Entry has been added with handle 1
RuntimeCmd:
```

Análisis de la salida: El compilador acepta el programa extendido sin errores adicionales. La CLI instala las seis entradas de tabla correctamente. Las claves de tcp_sessions se muestran como cuatro campos hexadecimales separados: 0a:00:00:01 = IP 10.0.0.1, 13:88 = puerto 5000 (= 0x1388), 0a:00:00:02 = IP 10.0.0.2, 27:0f = puerto 9999 (= 0x270F). La acción NoAction sin parámetros de tiempo de ejecución indica que el hit de la tabla simplemente permite el reenvío normal, que el bloque apply redirecciona a l2_exact_table.

4. **Paso 4 – Pruebas funcionales con nc y tcpdump**

   Objetivo: Validar experimentalmente que las conexiones TCP con puerto no registrado son descartadas silenciosamente y que la sesión registrada (10.0.0.1:5000 ↔ 10.0.0.2:9999) se establece con éxito.

   Comandos ejecutados (desde la CLI de Mininet):

   Se abren tres terminales de host:

   mininet> xterm h1 h2 h2

   En el primer terminal de h2, captura de tráfico TCP:

   (h2-1)# tcpdump -i h2-eth0 tcp -n

   En el segundo terminal de h2, servidor TCP persistente:

   (h2-2)# nc -lk -p 9999

   En el terminal de h1, primera prueba con puerto **no registrado** (6000):

   (h1)# nc -p 6000 -w 2 10.0.0.2 9999

   En el terminal de h1, segunda prueba con puerto **registrado** (5000):

   (h1)# nc -p 5000 10.0.0.2 9999

Comportamiento observado – conexión no autorizada (puerto 6000):

```
h1 nc -p 6000 -w 2 10.0.0.2 9999
[La conexion se cierra en h1 tras 2 segundos de espera]
```

Indicador clave: El terminal de h2 (tcpdump) permanece completamente vacío durante esta prueba. No aparece ningún paquete TCP en la interfaz h2-eth0, lo que confirma que el switch descartó el paquete SYN en el plano de datos antes de que pudiera alcanzar a h2.

Comportamiento observado – conexión autorizada (puerto 5000):

```
h1 nc -p 5000 10.0.0.2 9999
hola    [texto tecleado en h1]
h2-2 nc -lk -p 9999
hola    [mensaje recibido en h2]
```

Salida obtenida en tcpdump de h2 (conexión autorizada):

```
listening on h2-eth0, link-type EN10MB (Ethernet),
capture size 262144 bytes
22:28:07.816341 IP 10.0.0.1.5000 > 10.0.0.2.9999:
Flags [S], seq 2522455258,
  win 43440, options [mss 1460,...], length 0
22:28:07.836083 IP 10.0.0.2.9999 > 10.0.0.1.5000:
Flags [S.], seq 1586301920,
  ack 2522455259, win 43440, options [mss 1460,...],
  length 0
22:28:07.859952 IP 10.0.0.1.5000 > 10.0.0.2.9999:
Flags [.], ack 1, win 83,
  options [nop,nop,TS val 3679362583 ecr 360262583],
  length 0
22:28:12.815202 IP 10.0.0.1.5000 > 10.0.0.2.9999:
Flags [P.], seq 1:6, ack 1,
  win 83, length 5
```

```
22:28:12.820708 IP 10.0.0.2.9999 > 10.0.0.1.5000:
Flags [.], ack 6, win 85,
  length 0
```

Análisis de las salidas:

Conexión no autorizada: La ausencia total de tráfico en tcpdump es la confirmación definitiva del correcto funcionamiento del filtrado en el plano de datos. El nucleo de h1 intenta la conexión, genera un SYN desde el puerto 6000, pero el switch descarta el SYN porque la cuádrupla (10.0.0.1:6000, 10.0.0.2:9999) no tiene entrada en tcp_sessions. Tras el timeout de 2 segundos configurado con la opción -w 2, el cliente nc cierra la conexión.

Conexión autorizada: El tcpdump captura el proceso completo de establecimiento de tres vías (three-way handshake): (1) SYN de h1 con número de secuencia 2522455258, (2) SYN-ACK de h2 con número de secuencia 1586301920 y acuse de recibo 2522455259, y (3) ACK de h1. Tras el handshake, se intercambia un mensaje de 5 bytes (hola\n) que h2 acusa correctamente con el flag ACK y ack 6 (=1 inicial + 5 bytes de datos). El switch reenvía todos los paquetes de esta sesión porque la cuádrupla (10.0.0.1:5000, 10.0.0.2:9999) sí tiene entrada en tcp_sessions.

Explicación pedagógica: El contraste entre los dos resultados es la demostración más clara del mecanismo de control de sesiones: el criterio de autorización es exclusivamente la presencia de una entrada en la tabla, no el contenido ni el estado de la conexión. El estudiante observa cómo la tabla tcp_sessions actúa como una lista blanca (whitelist) de flujos autorizados, y cualquier flujo no incluido es rechazado en el propio plano de datos sin que el host destino llegue a recibir ningún paquete [6].

5. **Paso 5 – Verificación de conectividad general con pingall**

   Objetivo: Confirmar que el control de sesiones TCP no afecta el tráfico ICMP ni el tráfico no TCP entre los hosts.

   Concepto de red implicado: El bloque apply del ejercicio 2.2 sólo aplica tcp_sessions cuando hdr.tcp.isValid(), de modo que los paquetes ICMP (sin cabecera TCP) toman la rama else y son procesados por l2_exact_table con normalidad.

Comando ejecutado:

mininet> pingall

Salida obtenida:

```
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

Análisis: La conectividad ICMP no se ve afectada por la adición del control de sesiones TCP. Los paquetes ICMP son procesados exclusivamente por la rama de reenvío L2 sin pasar por tcp_sessions, lo que confirma que el pipeline distingue correctamente entre los diferentes tipos de tráfico según la validez de las cabeceras extraídas.

Explicación pedagógica: Este paso demuestra que las extensiones del pipeline P4 son modulares: al añadir funcionalidad de control TCP no se rompe el comportamiento existente de reenvío ARP e ICMP. Esta ortogonalidad es un principio de diseño deseable en P4, análoga a la composición de módulos en software design patterns [1].

Discusión: análisis comparativo de los tres ejercicios

El recorrido por los tres ejercicios permite trazar una progresión pedagógica clara que va desde la abstracción mínima del reenvío L2 hacia la implementación de funciones de control de sesiones a nivel de transporte.
En el ejercicio 1, el pipeline P4 es una traducción directa de la lógica de una tabla MAC (Forwarding Information Base) estática, idéntica funcionalmente a la de un switch OpenFlow con reglas preinstaladas. El valor pedagógico reside en observar que esta tabla es el resultado de un programa P4 compilado, no de un algoritmo de aprendizaje dinámico.
En el ejercicio 2, el pipeline adquiere capacidad de sintetizar tráfico de control: la acción arp_reply genera una trama ARP de respuesta a partir de una de

solicitud, sin invocar al plano de control ni al host destino. Este patrón de in-network processing es representativo de aplicaciones P4 de producción como proxies ARP, respuestas ICMP propias del switch, o cabeceras INT insertadas en tránsito [6].

En el ejercicio 2.2, el pipeline alcanza una lógica de filtrado orientado a sesiones: la tabla tcp_sessions actúa como una lista blanca de cuádruplas autorizadas, descartando cualquier flujo no registrado en el propio plano de datos. La diferencia fundamental con un firewall stateful es que este mecanismo P4 no mantiene estado de la conexión TCP (no registra si se completó el handshake, ni rastrea el estado de la sesión); es una decisión basada exclusivamente en las entradas de tabla instaladas por el plano de control.

Finalmente, los tres ejercicios también ilustran la arquitectura de P4 de extremo a extremo: la descripción del pipeline (código P4 compilado a JSON) permanece constante durante la ejecución, mientras que el plano de control (la CLI) modifica dinámicamente las entradas de tabla sin interrumpir el switch. Esta separación es la materialización concreta del principio SDN de separación control/datos en el contexto de planos de datos completamente programables [5].

Conclusiones pedagógicas

El laboratorio de introducción a P4 con ejercicios de ARP y control de sesiones TCP sitúa al estudiante frente al paradigma de planos de datos programables, donde el comportamiento del switch deja de ser una caja negra fija para convertirse en un programa sujeto a diseño, verificación y evolución [2]. A través de la traducción explícita de las RFC de ARP y TCP a cabeceras, parsers, tablas y acciones P4, el estudiante adquiere competencias en modelado de protocolos, diseño de pipelines y razonamiento sobre políticas de seguridad en el plano de datos, en coherencia con tendencias actuales en investigación y despliegue de redes programables [6].

La validación realizada confirma los siguientes resultados:

1. **Ejercicio 1**: el pipeline L2 con tabla de coincidencia exacta sobre MAC destino reenvía correctamente tramas entre h1 y h2 con 0 % de pérdida a

nivel de accesibilidad, verificado mediante pingall.

2. **Ejercicio 2**: el in-switch ARP responder intercepta y responde a las solicitudes ARP sin reenviarlas al host destino, confirmado mediante tcpdump en h2 (que no captura las solicitudes ARP de h1) y mediante el éxito del ping con cache ARP vacío.

3. **Ejercicio 2.2**: el control de sesiones TCP descarta silenciosamente los SYN no autorizados, confirmado por la ausencia de tráfico TCP en h2 para conexiones desde el puerto 6000, y permite correctamente la conexión registrada desde el puerto 5000, confirmado por el three-way handshake visible en tcpdump y la transferencia de datos bidireccional mediante nc.

Desde la perspectiva académica, la práctica es consistente con los tutoriales oficiales de P4.org y el curso SIGCOMM P4 Tutorial, que recomiendan empezar por ejercicios de parsing, reenvío básico y funciones elementales como ARP responder o filtros TCP antes de abordar tareas complejas como telemetría INT o integración de aprendizaje automático [8]. El alineamiento con los laboratorios previos del curso (Mininet-SDN, POX, Virtualización de Redes) garantiza una progresión pedagógica coherente en Internet del Futuro, desde la emulación de redes y el uso de controladores SDN hasta la programación directa de planos de datos con P4.

Referencias

[1] P4 Language Consortium, "P4$_{16}$ Language Specification, Version 1.2.x," 2019–2024. [Online]. Available: https://p4lang.github.io/p4-spec/docs/P4-16-v1.2.1.html

[2] J. F. Botero, "Data Plane Programmability – Introduction and Applications," Lecture notes, Universidad de Antioquia / University of Antwerp, 2020.

[3] D. C. Plummer, "An Ethernet Address Resolution Protocol," RFC 826, IETF, Nov. 1982 (updated 2024). [Online]. Available: https://www.rfc-editor.org/rfc/rfc826.html

[4] J. Postel, “Transmission Control Protocol,” RFC 793, IETF, Sep. 1981 (obsoleted by RFC 9293). [Online]. Available: https://www.rfc-editor.org/rfc/rfc793.html

[5] V. Haleplidis et al., “Software-Defined Networking (SDN): Layers and Architecture Terminology,” RFC 7426, IRTF SDNRG, Jan. 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7426.html

[6] C. Garzón, S. Ríos-Guiral, E. A. Leal, S. A. Gutiérrez, and J. F. Botero, “P4 Cybersecurity Solutions: Taxonomy and Open Challenges,” IEEE Access, vol. 12, pp. 6376–6399, 2024.

[7] P4.org / p4lang, “Behavioral Model v2 (BMv2): The Reference P4 Software Switch,” GitHub repository and documentation. [Online]. Available: https://github.com/p4lang/behavioral-model

[8] Stanford CS344, “P4 Tutorial and Lab Exercises,” 2021. [Online]. Available: https://2021-cs344.github.io/deliverables/tutorial-exercises/